

Trabajo Práctico de Estructura y Base de datos 2.603 – 10/06/2026

Sistema de Gestión de Torneos de Ajedrez

La Federación Internacional de Ajedrez (FIDE) nos ha encomendado el desafío de desarrollar una herramienta para procesar y analizar datos de torneos de ajedrez, brindando estadísticas una vez finalizada cada ronda. Como desarrolladores de software, nuestra misión es diseñar e implementar una aplicación en lenguaje C.

Archivos del Sistema

El sistema interactúa con los siguientes archivos para su operación:

config.txt (texto): Este archivo contiene la configuración global de la aplicación.

- Nombre de la Aplicación: Cadena de hasta 15 caracteres.
- Nombre del Torneo: Cadena de hasta 15 caracteres.
- Versión: Cadena de 5 caracteres (ej: "1.0").

jugadores.dat (binario): Almacena la información de cada participante.

- Número de jugador: Entero (int).
- Nombre: Cadena de hasta 15 caracteres.
- Apellido: Cadena de hasta 15 caracteres.
- País: Cadena de hasta 15 caracteres.

partidas.dat (binario): Registra el tiempo utilizado por cada jugador en cada ronda.

- Número de ronda: Entero (int).
- Número de jugador: Entero (int).
- Tiempo: Número de punto flotante (float) con 3 dígitos de precisión (ej: 90.123).

Requisitos Funcionales y Desarrollo

El programa deberá presentar un menú interactivo que ya fue desarrollado. La visualización del encabezado será la lectura del archivo config.txt y el menú tendrá las opciones 1 y 2 ya implementadas. Su tarea principal es desarrollar las opciones 3 y 4.

Formato del Encabezado del Menú (ya implementado):

[Nombre de la Aplicación] - [Nombre del Torneo] - Versión [X.X]

Menú Principal:

1. Opción 1: Mostrar jugadores (¡YA IMPLEMENTADO!)

Esta opción lista todos los jugadores registrados en el archivo jugadores.dat, mostrando su número, nombre, apellido y país.

2. Opción 2: Mostrar tiempos de carrera (¡YA IMPLEMENTADO!)

Esta opción lee y muestra todos los registros de vueltas contenidas en partidas.dat, agrupados por jugador, indicando el número de ronda, el número de jugador y el tiempo registrado.

3. Opción 3: Promedio de tiempo por jugador (A DESARROLLAR)

Solicitar al usuario que ingrese un número de jugador.

Validar que el número de jugador ingresado exista en el archivo jugadores.dat.

Si no existe, informarlo y permitir reingresarlo o volver al menú.

Si el jugador es válido, calcular el promedio de tiempo utilizado en sus partidas.

Mostrar en pantalla el nombre, apellido del jugador y su tiempo promedio.

4. Opción 4: Mejor tiempo de una ronda (A DESARROLLAR)

Solicitar al usuario que ingrese un número de ronda.

Si la ronda es válida, determinar que jugador tuvo el menor tiempo.

Mostrar en pantalla el nombre y apellido del jugador, país y su tiempo.

Consideraciones Generales:

- Debe utilizar manejo de archivos binarios y de texto según lo especificado.
- Implemente el manejo de errores al abrir archivos (verificar si son NULL).
- Utilice estructuras (struct) adecuadas para representar los datos de jugadores y partidas.
- Se valorará la claridad del código, y la correcta liberación de recursos (cierre de archivos).

El archivo de programa deberá ser renombrado de la siguiente manera:

P1_ApellidoNombre.c

Donde el o la estudiante deberá reemplazar con sus datos correspondientes y a continuación subirlo a la tarea que será habilitada a tal fin durante el tiempo que transcurre el examen.

A continuación se describe el código a partir del cual deben desarrollar las funciones solicitadas.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TAM 20

#define ARCH_PARTIDAS "partidas_ajedrez.dat"
#define ARCH_JUGADORES "jugadores.dat"
#define ARCH_CONFIG "config_ajedrez.txt"

typedef struct
{
    int numjugador;
    char nombre[16];
    char apellido[16];
    char pais[16];
} Jugador;

typedef struct
{
    int numronda;
    int numjugador;
    float tiempo;
} Partida;

void pausar();
void limpiarPantalla();

void mostrarVersion(char *aplicacion, char *torneo, char *version);
void MostrarJugadores();
void MostrarPartidas();
void MostrarPromedioTiempoJugador(); // opción 3
void MostrarMejorTiempoRonda();      // opción 4
void Menu();

int buscarJugador(int numero, Jugador *jugador);

int main()
{
    int opcion;

    Menu();
    printf("Ingrese la opcion deseada:\n");
    scanf("%d", &opcion);

    while (opcion != -1)
    {
        switch (opcion)
        {
            case 1:
                MostrarJugadores();
                pausar();
                break;

            case 2:
                MostrarPartidas();
```

```
        pausar();
        break;

    case 3:
        MostrarPromedioTiempoJugador();
        pausar();
        break;

    case 4:
        MostrarMejorTiempoRonda();
        pausar();
        break;

    default:
        printf("Opcion invalida.\n");
        pausar();
        break;
}

Menu();
printf("Ingrese la opcion deseada:\n");
scanf("%d", &opcion);
}

printf("\n\nSaliendo del sistema .....!!!\n\n");
pausar();

return 0;
}

void pausar()
{
#ifdef _WIN32
    system("pause");
#else
    printf("\nPresione ENTER para continuar...");
    getchar();
    getchar();
#endif
}

void limpiarPantalla()
{
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void mostrarVersion(char *aplicacion, char *torneo, char *version)
{
    FILE *archivo = fopen(ARCH_CONFIG, "r");

    if (archivo == NULL)
    {
        printf("El archivo %s no pudo abrirse.\n", ARCH_CONFIG);
        printf("Verifique que exista y que se encuentre en la misma carpeta del ejecutable.\n");
    }
    else
    {

```

```
fgets(aplicacion, 16, archivo);
fgets(torneo, 16, archivo);
fgets(version, 6, archivo);

printf("%s", aplicacion);
printf("%s", torneo);
printf("%s", version);

fclose(archivo);
}
}

void Menu()
{
    char aplicacion[16] = "";
    char torneo[16] = "";
    char version[6] = "";

    limpiarPantalla();

    mostrarVersion(aplicacion, torneo, version);

    printf("-----\n");
    printf("\tMENU DE OPCIONES\n");
    printf("1. Mostrar listado de jugadores.\n");
    printf("2. Mostrar listado de partidas.\n");
    printf("3. Mostrar promedio de tiempo por jugador.\n");
    printf("4. Mostrar mejor tiempo de una determinada ronda.\n");
    printf("Para salir pulsa << -1 >>.\n");
    printf("-----\n");
}

void MostrarJugadores()
{
    Jugador aux;
    FILE *archivo = fopen(ARCH_JUGADORES, "rb");

    if (archivo != NULL)
    {
        while (fread(&aux, sizeof(Jugador), 1, archivo) == 1)
        {
            printf("Numero de Jugador: %d| Nombre: %s. Apellido: %s. Pais: %s\n",
                aux.numjugador, aux.nombre, aux.apellido, aux.pais);
        }

        fclose(archivo);
    }
    else
    {
        printf("Error en la apertura del archivo %s\n", ARCH_JUGADORES);
    }
}

void MostrarPartidas()
{
    Partida aux;
    FILE *archivo = fopen(ARCH_PARTIDAS, "rb");

    if (archivo != NULL)
    {
        while (fread(&aux, sizeof(Partida), 1, archivo) == 1)
```

```
{
    printf("Numero de Ronda: %d, Numero de Jugador: %d, Tiempo: %.3f\n",
           aux.numronda, aux.numjugador, aux.tiempo);
}

fclose(archivo);
}
else
{
    printf("Error en la apertura del archivo %s\n", ARCH_PARTIDAS);
}
}

int buscarJugador(int numero, Jugador *jugador)
{
    FILE *archivo = fopen(ARCH_JUGADORES, "rb");
    Jugador aux;

    if (archivo == NULL)
    {
        printf("Error en la apertura del archivo %s\n", ARCH_JUGADORES);
        return 0;
    }

    while (fread(&aux, sizeof(Jugador), 1, archivo) == 1)
    {
        if (aux.numjugador == numero)
        {
            *jugador = aux;
            fclose(archivo);
            return 1;
        }
    }

    fclose(archivo);
    return 0;
}

void MostrarPromedioTiempoJugador()
{
    // COMPLETAR UDS.....
    printf("OPCION 03 A DESARROLLAR\n");
    system("pause");
}

void MostrarMejorTiempoRonda()
{
    // COMPLETAR UDS.....
    printf("OPCION 04 A DESARROLLAR\n");
    system("pause");
}
```